

Strings

String Data Type

- A string is a sequence of characters
- A string literal uses quotes
'Hello' or "Hello"
- For strings, + means “concatenate”
- When a string contains numbers, it is still a string
- We can convert numbers in a string into a number using `int()`

```
>>> str1 = "Hello"
>>> str2 = 'there'
>>> bob = str1 + str2
>>> print(bob)
Hellothere
>>> str3 = '123'
>>> str3 = str3 + 1
Traceback (most recent call
last):  File "<stdin>", line 1,
in <module>
TypeError: cannot concatenate
'str' and 'int' objects
>>> x = int(str3) + 1
>>> print(x)
124
>>>
```

Reading and Converting

- We prefer to read data in using **strings** and then parse and convert the data as we need
- This gives us more control over error situations and/or bad user input
- Input numbers must be **converted** from strings

```
>>> name = input('Enter: ')
Enter:Amit
>>> print(name)
Amit
>>> apple = input('Enter: ')
Enter:100
>>> x = apple - 10
Traceback (most recent call
last):  File "<stdin>", line 1,
in <module>
TypeError: unsupported operand
type(s) for -: 'str' and 'int'
>>> x = int(apple) - 10
>>> print(x)
90
```

Looking Inside Strings

- We can get at any single character in a string using an index specified in **square brackets**
- The index value must be an integer and starts at zero
- The index value can be an expression that is computed

b	a	n	a	n	a
0	1	2	3	4	5

```
>>> fruit = 'banana'
>>> letter = fruit[1]
>>> print(letter)
a
>>> x = 3
>>> w = fruit[x - 1]
>>> print(w)
n
```

A Character Too Far

- You will get a **python error** if you attempt to index beyond the end of a string
- So be careful when constructing index values and slices

```
>>> z = 'abc'
>>> print(z[5])
Traceback (most recent call
last):  File "<stdin>", line
1, in <module>
IndexError: string index out
of range
>>>
```

Strings Have Length

The built-in function `len` gives us the length of a string

b	a	n	a	n	a
0	1	2	3	4	5

```
>>> fruit = 'banana'
>>> print(len(fruit))
6
```

Looping Through Strings

Using a **while** statement, an **iteration variable**, and the **len** function, we can construct a loop to look at each of the letters in a string individually

```
fruit = 'banana'
index = 0
while index < len(fruit):
    letter = fruit[index]
    print(index, letter)
    index = index + 1
```

```
0 b
1 a
2 n
3 a
4 n
5 a
```

Looping Through Strings

- A definite loop using a **for** statement is much more elegant
- The **iteration variable** is completely taken care of by the **for** loop

```
fruit = 'banana'  
for letter in fruit:  
    print(letter)
```

b
a
n
a
n
a

Looping Through Strings

- A definite loop using a **for** statement is much more **elegant**
- The **iteration variable** is completely taken care of by the **for** loop

```
fruit = 'banana'
for letter in fruit :
    print(letter)
```

```
index = 0
while index < len(fruit) :
    letter = fruit[index]
    print(letter)
    index = index + 1
```

b
a
n
a
n
a

Looping and Counting

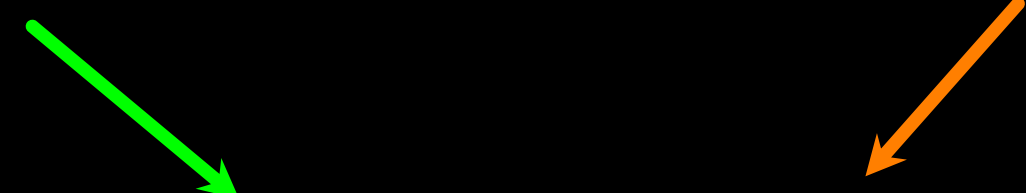
This is a simple loop that loops through each letter in a string and counts the number of times the loop encounters the 'a' character

```
word = 'banana'
count = 0
for letter in word :
    if letter == 'a' :
        count = count + 1
print(count)
```

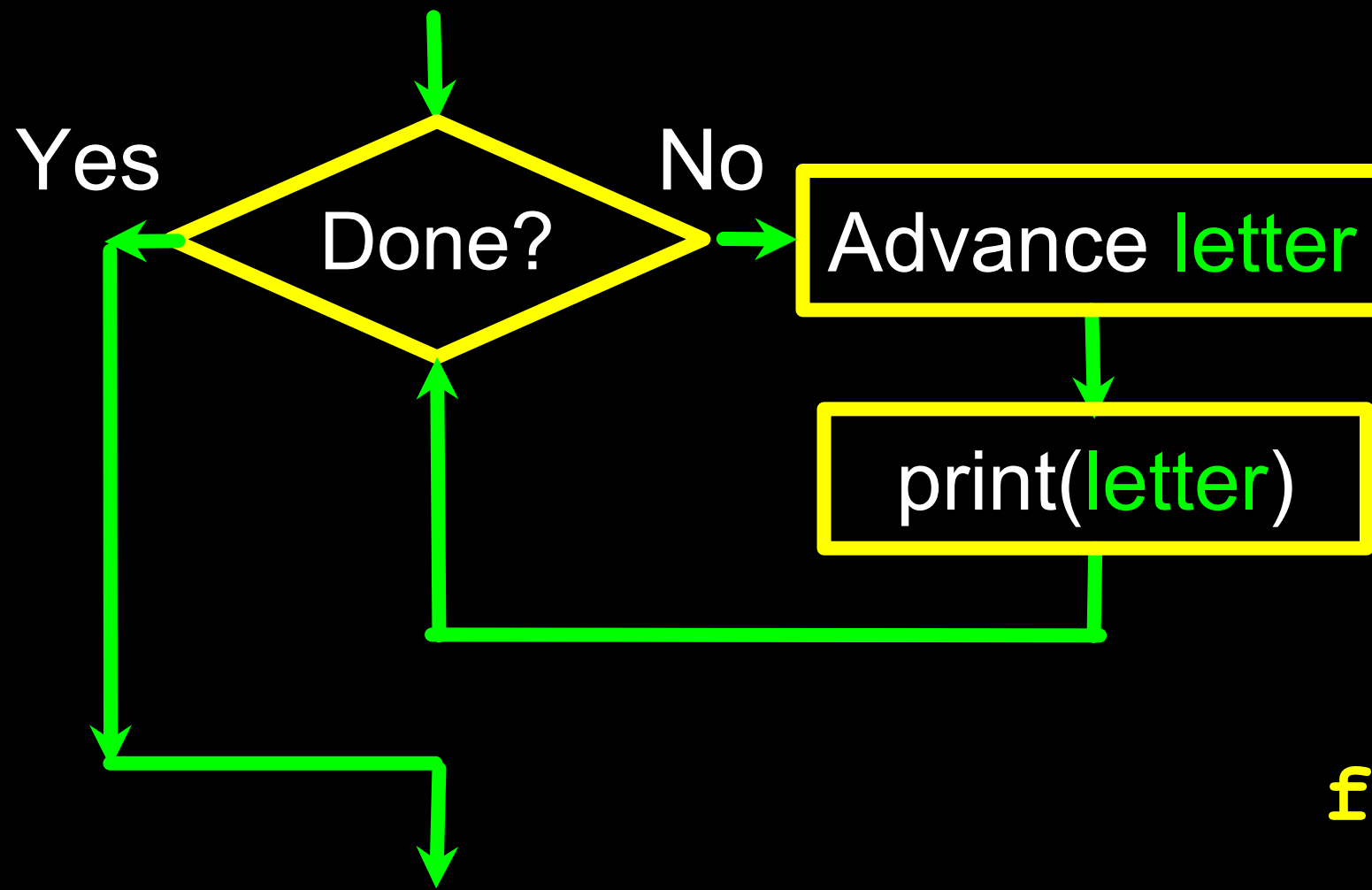
Looking Deeper into in

- The **iteration variable** “iterates” through the **sequence** (ordered set)
- The **block (body)** of code is executed once for each value **in** the **sequence**
- The **iteration variable** moves through all of the values **in** the **sequence**

Iteration variable Six-character string



```
for letter in 'banana' :  
    print(letter)
```



```
for letter in 'banana' :  
    print(letter)
```

The **iteration variable** “iterates” through the **string** and the **block (body)** of code is executed once for each value **in** the **sequence**

More String Operations

Strings and Character Numbers

The *ord* function returns the numeric (ordinal) code of a single character.

The *chr* function converts a numeric code to the corresponding character.

```
>>> ord("A")
```

```
65
```

```
>>> ord("a")
```

```
97
```

```
>>> chr(97)
```

```
'a'
```

```
>>> chr(65)
```

```
'A'
```

Slicing Strings

M	o	n	t	y		P	y	t	h	o	n
0	1	2	3	4	5	6	7	8	9	10	11

- We can also look at any continuous section of a string using a **colon operator**
- The second number is one beyond the end of the slice - “up to but not including”
- If the second number is beyond the end of the string, it stops at the end

```
>>> s = 'Monty Python'
>>> print(s[0:4])
Mont
>>> print(s[6:7])
P
>>> print(s[6:20])
Python
```

Slicing Strings

M	o	n	t	y		P	y	t	h	o	n
0	1	2	3	4	5	6	7	8	9	10	11

If we leave off the first number or the last number of the slice, it is assumed to be the beginning or end of the string respectively

```
>>> s = 'Monty Python'
>>> print(s[:2])
Mo
>>> print(s[8:])
thon
>>> print(s[:])
Monty Python
```

String Concatenation

When the `+` operator is applied to strings, it means “concatenation”

```
>>> a = 'Hello'
>>> b = a + 'There'
>>> print(b)
HelloThere
>>> c = a + ' ' + 'There'
>>> print(c)
Hello There
>>>
```

Using `in` as a Logical Operator

- The `in` keyword can also be used to check to see if one string is “in” another string
- The `in` expression is a logical expression that returns `True` or `False` and can be used in an `if` statement

```
>>> fruit = 'banana'
>>> 'n' in fruit
True
>>> 'm' in fruit
False
>>> 'nan' in fruit
True
>>> if 'a' in fruit :
...     print('Found it!')
...
Found it!
>>>
```

String Comparison

```
if word == 'banana':  
    print('All right, bananas.')  
if word < 'banana':  
    print('Your word, ' + word + ', comes before banana.')elif word > 'banana':  
    print('Your word, ' + word + ', comes after banana.')else:  
    print('All right, bananas.')
```

- Python has a number of string **functions** which are in the **string library**
- These **functions** are already **built into** every string - we invoke them by appending the function to the string variable
- These **functions** do not modify the original string, instead they return a new string that has been altered

String Library

```
>>> s1 = 'Hello Bob'
>>> z = s1.lower()
>>> print(z)
hello bob
>>> print(s1)
Hello Bob
>>> print('Hi There'.lower())
hi there
>>>
```

```
>>> s1 = 'Hello world'
>>> type(s1)
<class 'str'>
>>> dir(s1)
['capitalize', 'casefold', 'center', 'count', 'encode',
'endswith', 'expandtabs', 'find', 'format', 'format_map',
'index', 'isalnum', 'isalpha', 'isdecimal', 'isdigit',
'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace',
'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip',
'maketrans', 'partition', 'replace', 'rfind', 'rindex', 'rjust',
'rstrip', 'rsplit', 'rstrip', 'split', 'splitlines',
'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper',
'zfill']
```

<https://docs.python.org/3/library/stdtypes.html#string-methods>

str.replace(*old*, *new*[, *count*])

Return a copy of the string with all occurrences of substring *old* replaced by *new*. If the optional argument *count* is given, only the first *count* occurrences are replaced.

str.rfind(*sub*[, *start*[, *end*]])

Return the highest index in the string where substring *sub* is found, such that *sub* is contained within *s*[*start*:*end*]. Optional arguments *start* and *end* are interpreted as in slice notation. Return `-1` on failure.

str.rindex(*sub*[, *start*[, *end*]])

Like `rfind()` but raises `ValueError` when the substring *sub* is not found.

str.rjust(*width*[, *fillchar*])

Return the string right justified in a string of length *width*. Padding is done using the specified *fillchar* (default is an ASCII space). The original string is returned if *width* is less than or equal to `len(s)`.

str.rpartition(*sep*)

Split the string at the last occurrence of *sep*, and return a 3-tuple containing the part before the separator, the separator itself, and the part after the separator. If the separator is not found, return a 3-tuple containing two empty strings, followed by the string itself.

str.rsplit(*sep*=None, *maxsplit*=-1)

Return a list of the words in the string, using *sep* as the delimiter string. If *maxsplit* is given, at most *maxsplit* splits are done, the *rightmost* ones. If *sep* is not specified or `None`, any whitespace string is a separator. Except for splitting from the right, `rsplit()` behaves like `split()` which is described in detail below.

More string functionality

```
>>> len("GATTACA")  
7
```

←Length

```
>>> "GAT" + "TACA"  
'GATTACA'
```

←Concatenation

```
>>> "A" * 10  
'AAAAAAAAAA'
```

←Repeat

```
>>> "GAT" in "GATTACA"  
True
```

←Substring test

```
>>> "AGT" in "GATTACA"  
False
```

String methods

In Python, a method is a function that is defined with respect to a particular object.

The syntax is `<object>.<method>(<parameters>)`

```
>>> dna = "ACGT"
```

```
>>> dna.find("T")
```

```
3
```

String methods

```
>>> "GATTACA".find("ATT")
```

```
1
```

```
>>> "GATTACA".count("T")
```

```
2
```

```
>>> "GATTACA".lower()
```

```
'gattaca'
```

```
>>> "gattaca".upper()
```

```
'GATTACA'
```

```
>>> "GATTACA".replace("G", "U")
```

```
'UATTACA'
```

```
>>> "GATTACA".replace("AT", "***")
```

```
'G**TACA'
```

```
>>> "GATTACA".startswith("G")
```

```
True
```

```
>>> "GATTACA".startswith("g")
```

```
False
```

Strings are immutable

Strings cannot be modified; instead, create a new one.

```
>>> s = "GATTACA"
```

```
>>> s[3] = "C"
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in ?
```

```
TypeError: object doesn't support item assignment
```

```
>>> s = s[:3] + "C" + s[4:]
```

```
>>> s  
'GATCACA'
```

```
>>> s = s.replace("G", "U")
```

```
>>> s  
'UATCACA'
```

Strings are immutable

- String methods do not modify the string; they return a new string.

```
>>> sequence = "ACGT"
```

```
>>> sequence.replace("A", "G")
```

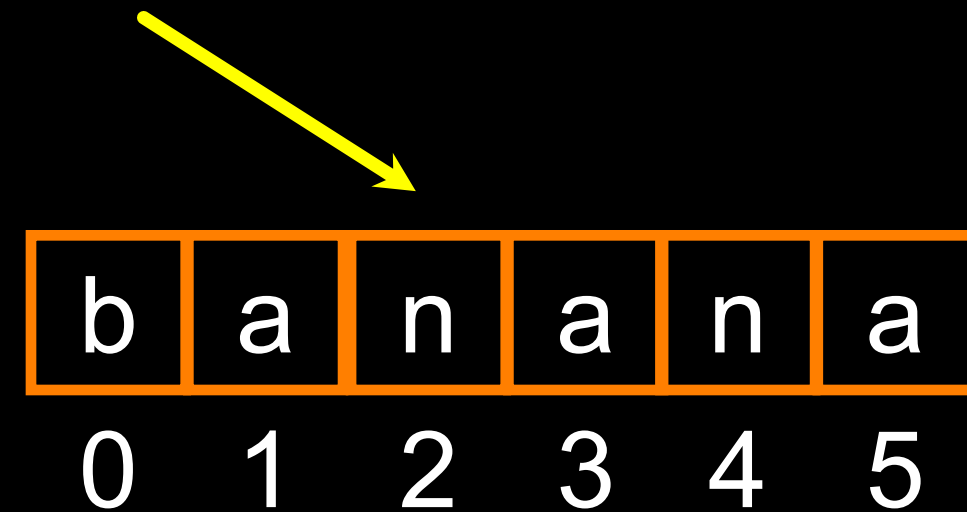
```
'GCGT'
```

```
>>> print sequence
```

```
ACGT
```

Searching a String

- We use the `find()` function to search for a substring within another string
- `find()` finds the first occurrence of the substring
- If the substring is not found, `find()` returns `-1`
- Remember that string position starts at zero



```
>>> fruit = 'banana'
>>> pos = fruit.find('na')
>>> print(pos)
2
>>> aa = fruit.find('z')
>>> print(aa)
-1
```

Making everything UPPER CASE

- You can make a copy of a string in **lower case** or **upper case**
- Often when we are searching for a string using **find()** we first convert the string to lower case so we can search a string regardless of case

```
>>> s1 = 'Hello Bob'
>>> n = s1.upper()
>>> print(n)
HELLO BOB
>>> w = s1.lower()
>>> print(w)
hello bob
>>>
```

Search and Replace

- The `replace()` function is like a “search and replace” operation in a word processor
- It replaces **all occurrences** of the **search string** with the **replacement string**

```
>>> s = 'Hello Bob'
>>> n = s.replace('Bob', 'Jane')
>>> print(n)
Hello Jane
>>> n = s.replace('o', 'x')
>>> print(n)
Hel1x Bxb
>>>
```

Stripping Whitespace

- Sometimes we want to take a string and remove whitespace at the beginning and/or end
- `lstrip()` and `rstrip()` remove whitespace at the left or right
- `strip()` removes both beginning and ending whitespace

```
>>> s1 = '    Hello Bob    '  
>>> s1.lstrip()  
'Hello Bob '  
>>> s1.rstrip()  
'    Hello Bob'  
>>> s1.strip()  
'Hello Bob '  
>>>
```

Prefixes

```
>>> line = 'Please have a nice day'
>>> line.startswith('Please')
True
>>> line.startswith('p')
False
```

Parsing and Extracting

21 31
↓ ↓
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2020

uct.ac.za

```
>>> data = 'From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2020'
>>> atpos = data.find('@')
>>> print(atpos)
21
>>> sppos = data.find(' ', atpos)
>>> print(sppos)
31
>>> host = data[atpos+1 : sppos]
>>> print(host)
uct.ac.za
```

Other String Methods

There are a number of other string methods.

- `s.capitalize()` – Copy of `s` with only the first character capitalized
- `s.title()` – Copy of `s`; first character of each word capitalized
- `s.center(width)` – Center `s` in a field of given width

```
>>> s="good morning"
>>> s.capitalize()
'Good morning'
```

```
>>> s="good morning"
>>> s.title()
'Good Morning'
```

```
>>> s="good morning"
>>> s.center(20)
'   good morning   '
```

Other String Operations

- `s.count(sub)` – Count the number of occurrences of `sub` in `s`
- `s.find(sub)` – Find the first position where `sub` occurs in `s`
- `s.join(list)` – Concatenate list of strings into one large string using `s` as separator.
- `s.ljust(width)` – Like `center`, but `s` is left-justified

```
>>> s="good morning"
>>> s.count('o')
3
```

```
>>> s="good morning"
>>> s.find('d')
3
```

```
>>> s.ljust(20)
'good morning      '
```

String Formatting

`%d` is used for formatting integer data

```
>>> a=int(input("Enter integer value:"))
```

```
Enter integer value:20
```

```
>>> print("Integer value is: %d" %a)
```

```
Integer value is: 20
```

String Formatting

`%f` is used for formatting floating point data

```
>>> a=float(input("Enter float value:"))
```

```
Enter float value:12.253
```

```
>>> print("Integer value is: %f" %a)
```

```
Integer value is: 12.253000
```

String Formatting

`%.<size>f` is used for formatting floating point data with fixed size of fractional number

```
>>> a=float(input("Enter float value:"))
```

```
Enter float value:12.45
```

```
>>> print("float value is: %f" %a)
```

```
float value is: 12.450000
```

```
>>> print("float value is: %.2f" %a)
```

```
float value is: 12.45
```

String Formatting

`%s` is used for formatting string objects

```
1.# Define a string value  
2.str = "Python"  
3.# using %s to appending string  
4.print("Hello! Welcome to, %s" % str)
```

Output: Hello! Welcome to, Python

String Formatting

A string with one or more placeholders is needed to begin using the `format()` method. Placeholders are set apart by wavy supports, with discretionary positional or named contentions inside.

```
1.name = "John"  
2.age = 30  
3.formatted_string = "My name is {} and I'm {} years old.".format(name, age)  
4.print(formatted_string)
```

Output:

My name is John and I'm 30 years old.

Acknowledgements / Contributions



These slides are Copyright 2010- Charles R. Severance (www.dr-chuck.com) of the University of Michigan School of Information and open.umich.edu and made available under a Creative Commons Attribution 4.0 License. Please maintain this last slide in all copies of the document to comply with the attribution requirements of the license. If you make a change, feel free to add your name and organization to the list of contributors on this page as you republish the materials.

Initial Development: Charles Severance, University of Michigan School of Information